

Universidad San Carlos de Guatemala
Centro Universitario de Occidente
Ingenieria en Sistemas - Inteligencia Artificial 1



MANUAL TECNICO

Sistema Generador de Horarios Personalizados con Algoritmo Genetico

Estudiantes

Carnet	Nombre
202030556	Javier Isaías Coyoy Marín
201430801	Luis Basilio Baquiaux Sic

Indice

1. Introduccion	3
2. Tecnologias Utilizadas	3
2.1 Backend	3
2.2 Frontend	3
3. Arquitectura del Backend	4
3.1 Estructura de Directorios	4
3.2 Descripcion de Capas	4
Capa de Rutas (routes/)	4
Capa de Controladores (controllers/)	5
Capa de Servicios (services/)	5
Capa de Modelos (models/)	5
Modulo AlgoritmoGenetico	5
3.3 Componentes del Algoritmo Genetico	5
4. Endpoints de la API	5
4.1 Autenticacion - /api/auth	6
4.2 Estudiantes - /api/estudiantes	6
4.3 Pensum - /api/pensum	6
4.4 Dashboard - /api/dashboard	6
4.5 Carga de Datos - /api/carga-datos	6
4.6 Algoritmo Genetico - /api/ga	7
5. Base de Datos	7
5.1 Descripcion General	7
5.2 Diagrama Tablas	7
5.3 Script de Creacion	9
5.4 Descripcion de Tablas	11
Grupo 1: Ubicacion Geografica	11
Grupo 2: Datos Maestros de Carrera y Estudiante	12
Grupo 3: Informacion de Contacto	12
Grupo 4: Estructura Academica	12
Grupo 5: Registro y Control	12
5.5 Relaciones Clave	13
6. Arquitectura del Frontend	13
6.1 Estructura de Directorios	13
6.2 Vistas Principales	13
7. Flujo de Generacion de Horario	14
8. Configuracion e Instalacion	14
8.1 Variables de Entorno (Backend)	14
8.2 Comandos de Instalacion	15
8.3 Requisitos del Sistema	15
9. Manejo de Errores	15

1. Introduccion

El presente manual tecnico documenta la arquitectura, estructura de base de datos, endpoints y tecnologias empleadas en el sistema Generador de Horarios Personalizados. Este sistema permite a estudiantes universitarios seleccionar cursos disponibles segun su pensum y recibir un horario optimizado generado mediante un Algoritmo Genetico (GA).

El sistema consta de dos componentes principales: un backend desarrollado en Node.js con Express y TypeScript, y un frontend construido con Vue 3, Vuetify 4 y TypeScript. La comunicacion entre ambas capas se realiza mediante una API REST.

2. Tecnologias Utilizadas

2.1 Backend

Tecnologia / Libreria	Version	Descripcion
Node.js / TypeScript	^5.9.3	Entorno de ejecucion y tipado estatico
Express	^5.1.0	Framework HTTP para la API REST
Sequelize	^6.37.7	ORM para manejo de base de datos relacional
mysql2 / pg	^3.15.3 / ^8.13.3	Drivers de base de datos MySQL y PostgreSQL
genetic-js	^0.1.14	Libreria base para el algoritmo genetico
multer	^2.0.2	Manejo de carga de archivos (CSV)
csv-parse	^6.1.0	Parseo de archivos CSV importados
express-async-handler	^1.2.0	Manejo de errores asincronos en controladores
swagger-ui-express	^5.0.1	Documentacion interactiva de la API
dotenv	^17.2.3	Gestion de variables de entorno
axios	^1.13.2	Cliente HTTP para consumo de servicios externos

2.2 Frontend

Tecnologia / Libreria	Version	Descripcion
Vue 3	^3.5.31	Framework progresivo para interfaces reactivas
Vuetify 4	^4.0.1	Libreria de componentes Material Design para Vue

Vue Router	^5.0.4	Enrutamiento SPA del lado del cliente
Pinia	^3.0.4	Gestion de estado global reactivo
TypeScript	~6.0.0	Tipado estatico para mayor robustez del codigo
Vite	^8.0.3	Bundler y servidor de desarrollo ultrarapido
@mdi/font	^7.4.47	Iconografia Material Design Icons

3. Arquitectura del Backend

El backend sigue una arquitectura en capas (Layered Architecture) que separa responsabilidades en: rutas, controladores, servicios, modelos y la capa especializada del Algoritmo Genetico. El punto de entrada es `src/index.ts`, que instancia la clase `Server`.

3.1 Estructura de Directorios

```

src/
├── AlgoritmoGenetico/    # Modulo autocontenido del GA
│   ├── genetic/         # Componentes del algoritmo
│   │   ├── AlgoritmoGenetico.ts
│   │   ├── Cruce.ts
│   │   ├── FuncionAptitud.ts
│   │   ├── Mutacion.ts
│   │   ├── Poblacion.ts
│   │   └── Seleccion.ts
│   ├── mappers/         # Transformacion de datos
│   ├── services/        # GaService.ts
│   ├── types/           # Tipos e interfaces del GA
│   └── utils/           # Validadores de restricciones
├── config/              # Configuracion de BD y Swagger
├── controllers/         # Controladores HTTP
├── database/            # Conexion y sincronizacion ORM
├── exceptions/          # Excepciones personalizadas
├── middlewares/         # Error handler y upload
├── models/              # Modelos Sequelize
├── routes/              # Definicion de rutas Express
├── services/            # Logica de negocio
├── server/Server.ts     # Clase servidor principal
└── types/               # Tipos globales

```

3.2 Descripcion de Capas

Capa de Rutas (routes/)

Define los endpoints de cada dominio y los asocia con sus controladores correspondientes. No contiene logica de negocio.

Capa de Controladores (controllers/)

Recibe las peticiones HTTP, valida parametros basicos, invoca al servicio correspondiente y retorna la respuesta JSON. Usa express-async-handler para propagar errores al middleware centralizado.

Capa de Servicios (services/)

Contiene la logica de negocio: consultas ORM, reglas de importacion CSV, calculo de datos del dashboard y orquestacion del algoritmo genetico.

Capa de Modelos (models/)

Define los modelos Sequelize que representan las tablas de la base de datos, incluyendo sus relaciones (hasMany, belongsTo, etc.).

Modulo AlgoritmoGenetico

Es un modulo autocontenido con sus propios tipos, servicios, mappers y componentes geneticos. GaService.ts orquesta la ejecucion del GA, recibiendo la seleccion del estudiante y retornando el horario optimizado.

3.3 Componentes del Algoritmo Genetico

Componente	Descripcion
AlgoritmoGenetico.ts	Orquesta el ciclo evolutivo: poblacion inicial, evaluacion, seleccion, cruce y mutacion.
Poblacion.ts	Genera y mantiene el conjunto de individuos (horarios candidatos).
FuncionAptitud.ts	Calcula el fitness de cada individuo segun restricciones de horario y pensum.
Seleccion.ts	Implementa el mecanismo de seleccion de padres (torneo, ruleta, etc.).
Cruce.ts	Define el operador de cruzamiento para generar nuevos individuos.
Mutacion.ts	Aplica mutaciones aleatorias para mantener diversidad genetica.
ValidadorRestricciones.ts	Verifica choques de horario, prerequisites y creditos maximos.
ValidatorPensum.ts	Valida que los cursos seleccionados correspondan al pensum del estudiante.
MapperFromHorarioGeneral.ts	Transforma el HorarioGeneral al formato de individuo del GA.

4. Endpoints de la API

El servidor corre por defecto en el puerto 8080. Todos los endpoints estan bajo el prefijo /api.

4.1 Autenticacion - /api/auth

Metodo	Ruta	Descripcion
POST	/api/auth/login	Autentica a un estudiante con carnet y contrasenia. Retorna datos del estudiante.

Body requerido:

```
{ "carnet": 202312345, "contrasenia": "pass" }
```

4.2 Estudiantes - /api/estudiantes

Metodo	Ruta	Descripcion
GET	/api/estudiantes/:carnet	Obtiene los datos personales del estudiante por carnet.
PUT	/api/estudiantes/:carnet	Actualiza datos personales (dpi, nombres, apellidos, fecha_nacimiento).
PUT	/api/estudiantes/:carnet/contacto	Actualiza informacion de contacto (municipio, direccion, correo, telefono).

4.3 Pensum - /api/pensum

Metodo	Ruta	Descripcion
GET	/api/pensum?carnet=X	Retorna el pensum completo del estudiante (cursos, prerequisites, historial, estado).

4.4 Dashboard - /api/dashboard

Metodo	Ruta	Descripcion
GET	/api/dashboard?carnet=X	Retorna estadisticas academicas del estudiante: creditos aprobados, cursos pendientes, rendimiento.

4.5 Carga de Datos - /api/carga-datos

Metodo	Ruta	Descripcion
POST	/api/carga-datos/pensum	Importa pensum y prerequisites desde dos archivos CSV (multipart/form-data): campos 'pensum' y 'prerequisito'.

POST	/api/carga-datos/extras	Importa estudiantes, historial y contacto desde tres archivos CSV: campos 'estudiante', 'historial' y 'contacto'.
------	-------------------------	---

4.6 Algoritmo Genetico - /api/ga

Metodo	Ruta	Descripcion
POST	/api/ga/generar-horario	Ejecuta el algoritmo genetico y retorna el horario personalizado optimizado para el estudiante.

Body requerido:

```
{
  "carnet": 202312345,
  "obligatorios": [{ "codigo": 101, "nombre": "Matematica 1", "creditos": 4 }],
  "opcionales": [{ "codigo": 205, "nombre": "Electiva A", "creditos": 3 }]
}
```

5. Base de Datos

5.1 Descripcion General

La base de datos sigue un modelo relacional normalizado, dividida en cinco grupos funcionales que soportan los flujos del sistema: ubicacion geografica, datos maestros de carrera y estudiante, informacion de contacto, estructura academica (cursos y pensum) y registro historico (historial academico y horarios asignados).

Compatible con MySQL y PostgreSQL (configurado via drivers mysql2 y pg). El ORM Sequelize gestiona la sincronizacion automatica de modelos.

5.2 Diagrama Tablas



5.3 Script de Creacion

-- 1. Tablas de Ubicación Geográfica

```
CREATE TABLE Pais (  
id INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE Departamento (  
id INT PRIMARY KEY,  
pais_id INT,  
nombre VARCHAR(100) NOT NULL,  
FOREIGN KEY (pais_id) REFERENCES Pais(id)  
);
```

```
CREATE TABLE Municipio (  
id INT PRIMARY KEY,  
departamento_id INT,  
nombre VARCHAR(100) NOT NULL,  
FOREIGN KEY (departamento_id) REFERENCES Departamento(id)  
);
```

-- 2. Tablas Maestras de Carrera y Estudiante

```
CREATE TABLE Carrera (  
id INT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
pensum VARCHAR(50),  
credito_obligatorios INT,  
credito_opcionales INT  
);
```

```
CREATE TABLE Tipo_Estudiante (  
id INT PRIMARY KEY,  
nombre VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Estudiante (  
carnet INT PRIMARY KEY,  
carrera_id INT,  
tipo_estudiante INT,
```

```
dpi VARCHAR(20),
nombres VARCHAR(100),
apellidos VARCHAR(100),
fecha_nacimiento DATE,
contrasenia VARCHAR(255),
FOREIGN KEY (carrera_id) REFERENCES Carrera(id),
FOREIGN KEY (tipo_estudiante) REFERENCES Tipo_Estudiante(id)
);
```

```
-- 3. Información de Contacto
CREATE TABLE Informacion_Contacto_Est (
id INT PRIMARY KEY,
carnet_estudiante_id INT,
municipio_vivienda_id INT,
direccion VARCHAR(255),
correo_institucional VARCHAR(100),
telefono VARCHAR(20),
FOREIGN KEY (carnet_estudiante_id) REFERENCES Estudiante(carnet),
FOREIGN KEY (municipio_vivienda_id) REFERENCES Municipio(id)
);
```

```
-- 4. Tablas de Cursos y Estructura Académica
CREATE TABLE Curso (
codigo INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
creditos INT,
semestre INT,
es_obligatorio BOOLEAN
);
```

```
CREATE TABLE Carrera_Cursos (
id INT PRIMARY KEY,
carrera_id INT,
codigo_curso_id INT,
FOREIGN KEY (carrera_id) REFERENCES Carrera(id),
FOREIGN KEY (codigo_curso_id) REFERENCES Curso(codigo)
);
```

```
CREATE TABLE Prerequisito (
id INT PRIMARY KEY,
```

```

codigo_curso_id INT,
codigo_curso_prre INT,
FOREIGN KEY (codigo_curso_id) REFERENCES Curso(codigo),
FOREIGN KEY (codigo_curso_prre) REFERENCES Curso(codigo)
);

```

```

-- 5. Tablas de Registro y Control
CREATE TABLE Historial (
id INT PRIMARY KEY,
carnet_estudiante_id INT,
codigo_curso_id INT,
nota INT,
aprobado BOOLEAN,
es_semestre VARCHAR(20),
fecha_registro DATE,
FOREIGN KEY (carnet_estudiante_id) REFERENCES Estudiante(carnet),
FOREIGN KEY (codigo_curso_id) REFERENCES Curso(codigo)
);

```

```

CREATE TABLE Horario (
id INT PRIMARY KEY,
carnet_estudiante_id INT,
codigo_curso_id INT,
seccion VARCHAR(5),
docente VARCHAR(100),
horario VARCHAR(50),
salon VARCHAR(20),
FOREIGN KEY (carnet_estudiante_id) REFERENCES Estudiante(carnet),
FOREIGN KEY (codigo_curso_id) REFERENCES Curso(codigo)
);

```

5.4 Descripcion de Tablas

Grupo 1: Ubicacion Geografica

Tabla	PK	Descripcion
Pais	id	Catalogo de paises. Raiz de la jerarquia geografica.
Departamento	id	Departamentos o estados vinculados a un pais mediante pais_id.

Municipio	id	Municipios vinculados a un departamento mediante departamento_id.
-----------	----	---

Grupo 2: Datos Maestros de Carrera y Estudiante

Tabla	PK	Descripcion
Carrera	id	Registra las carreras disponibles con su pensum, creditos obligatorios y opcionales requeridos para graduacion.
Tipo_Estudiante	id	Catalogo de tipos de estudiante (regular, traslado, extranjero, etc.).
Estudiante	carnet	Entidad central del sistema. Almacena datos personales, carrera, tipo de estudiante y credenciales de acceso (contrasenia hasheada).

Grupo 3: Informacion de Contacto

Tabla	PK	Descripcion
Informacion_Contacto_Est	id	Extiende al Estudiante con datos de contacto: municipio de vivienda, direccion fisica, correo institucional y telefono. Relacion 1:1 con Estudiante.

Grupo 4: Estructura Academica

Tabla	PK	Descripcion
Curso	codigo	Catalogo maestro de cursos con codigo, nombre, creditos, semestre al que pertenece y si es obligatorio u opcional.
Carrera_Cursos	id	Tabla pivotte que asocia cursos con carreras (relacion N:M). Permite que un curso pertenezca a multiples carreras.
Prerequisito	id	Define dependencias entre cursos: codigo_curso_id requiere aprobar codigo_curso_prre antes de poder cursarse. Autoreferencia sobre Curso.

Grupo 5: Registro y Control

Tabla	PK	Descripcion
Historial	id	Registra el historial academico del estudiante: curso cursado, nota obtenida, si fue aprobado, semestre y fecha de registro. Insumo principal para el GA.
Horario	id	Almacena los horarios asignados al estudiante: seccion, docente, franja horaria y salon. Generado por el Algoritmo Genetico y persistido en esta tabla.

5.5 Relaciones Clave

- Estudiante -> Carrera (N:1): cada estudiante pertenece a una carrera.
- Estudiante -> Tipo_Estudiante (N:1): clasificacion del estudiante.
- Informacion_Contacto_Est -> Estudiante (1:1): datos de contacto extensos.
- Informacion_Contacto_Est -> Municipio (N:1): ubicacion de vivienda.
- Carrera_Cursos (N:M): asocia Carrera con Curso permitiendo pensum multi-carrera.
- Prerequisito (auto-referencia sobre Curso): grafo dirigido de dependencias.
- Historial -> Estudiante, Curso: trazabilidad academica completa.
- Horario -> Estudiante, Curso: resultado persistido del algoritmo genetico.

6. Arquitectura del Frontend

El frontend es una Single Page Application (SPA) construida con Vue 3 (Composition API), Vuetify 4 como libreria de UI y Pinia para gestion de estado. La comunicacion con el backend se realiza a traves de una capa de servicios con Axios (via api.ts).

6.1 Estructura de Directorios

```
src/
├── components/           # Componentes reutilizables (BarraNavegacion.vue)
├── plugins/vuetify.ts    # Configuracion de Vuetify y temas
├── router/index.ts       # Definicion de rutas de la SPA
├── services/             # Capa de acceso a la API
│   ├── api.ts           # Instancia Axios con baseURL y headers
│   ├── auth/            # Login y gestion de sesion
│   ├── cargaDatos/      # Importacion de archivos CSV
│   ├── dashboard/       # Estadisticas academicas
│   ├── estudiante/      # Perfil y actualizacion de datos
│   ├── horario/         # Consumo del GA y visualizacion
│   ├── pensum/          # Consulta del pensum del estudiante
│   └── types/index.ts    # Tipos e interfaces TypeScript globales
├── types/index.ts        # Tipos e interfaces TypeScript globales
└── views/               # Vistas principales de la aplicacion
```

6.2 Vistas Principales

Vista	Descripcion
LoginView.vue / IniciarSesionView.vue	Formulario de inicio de sesion. Valida carnet y contrasenia contra /api/auth/login.
DashboardView.vue	Muestra estadisticas academicas del estudiante: creditos, cursos aprobados y rendimiento.
PensumView.vue	Visualiza el pensum del estudiante con estado de cada curso (aprobado, pendiente, bloqueado).

EstudianteView.vue	Permite visualizar y editar datos personales y de contacto del estudiante.
HorarioEstudianteView.vue	Permite seleccionar cursos obligatorios y opcionales, invocar el GA y visualizar el horario resultante.
CargaDatosView.vue	Interfaz administrativa para importar archivos CSV de pensum, prerequisites, estudiantes e historial.

7. Flujo de Generacion de Horario

A continuacion se describe el flujo end-to-end del proceso central del sistema:

1. El estudiante inicia sesion con su carnet y contrasenia (POST /api/auth/login).
2. Consulta su pensum disponible (GET /api/pensum?carnet=X) para ver cursos aptos.
3. En HorarioEstudianteView selecciona los cursos obligatorios y opcionales que desea cursar.
4. El frontend envia la seleccion al backend (POST /api/ga/generar-horario).
5. GaService.ts prepara el contexto: carga el HorarioGeneral y mapea los datos al formato del GA.
6. El AlgoritmoGenetico ejecuta la evolucion: inicializa poblacion, evalua fitness, selecciona, cruza y muta durante N generaciones.
7. FuncionAptitud.ts penaliza conflictos de horario, prerequisites no cumplidos y sobrecarga de creditos.
8. El mejor individuo (horario optimo) es retornado como ResultadoGA y enviado al frontend.
9. HorarioEstudianteView renderiza el horario personalizado al estudiante.

8. Configuracion e Instalacion

8.1 Variables de Entorno (Backend)

```
# PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_NAME=db_sistema_horarios
DB_USER=user_db
DB_PASSWORD=password

# Servidor
PORT=3001
NODE_ENV=development
```

```
# Generador horarios api  
GA_API_URL=http://localhost:3000
```

8.2 Comandos de Instalacion

Backend:

```
npm install  
npm run dev      # Desarrollo con nodemon  
npm run build    # Compilar TypeScript  
npm start        # Produccion
```

Frontend:

```
npm install  
npm run dev      # Servidor de desarrollo Vite  
npm run build    # Build de produccion
```

8.3 Requisitos del Sistema

- Node.js $\geq 20.19.0$ o $\geq 22.12.0$
- MySQL ≥ 8.0 o PostgreSQL ≥ 14
- npm ≥ 9

9. Manejo de Errores

El backend implementa un middleware centralizado de errores (errorHandler.ts) que captura excepciones lanzadas desde controladores y servicios. Las excepciones personalizadas se definen en ApiExceptions.ts, incluyendo BadRequestError, NotFoundError y otras, cada una con su código HTTP correspondiente.

En el frontend, los servicios interceptan respuestas de error HTTP y propagan mensajes al usuario mediante componentes Vuetify (snackbars, alertas).